

# Programação Orientada a Objetos

---

- ✓ ENCAPSULAMENTO DE DADOS
- ✓ MODIFICADORES DE ACESSO
- ✓ HERANÇA
- ✓ INTERFACES



# Na aula passada...

---

## Estruturas de repetição

- ✓ Também conhecidas como laços (loop).
- ✓ Utilizadas para executar repetidamente uma instrução ou um bloco de instruções enquanto uma determinada condição for verdadeira.
- ✓ As estruturas de repetição em Java são:
  - `for`
  - `while`
  - `do ... while`



# Na aula passada...

## Estruturas de repetição

Para determinarmos qual é a estrutura mais adequada, devemos saber:

- ✔ o número de vezes que o trecho programa vai ser executado: laços contados
- ✔ ou a condição para que ela aconteça: laços condicionais

Laços contados: um **contador** irá auxiliar no laço. Neste laço, a repetição da estrutura repete-se até que o contador atinja o limite estipulado na condição.

Laços condicionais: o valor é desconhecido e devemos utilizar uma variável com valor predefinido em uma condição dentro do laço para finalizarmos a repetição



# Na aula passada...

---

## Estruturas de repetição

Independente do tipo de laço, todos são constituídos de três partes:

- ✓ Inicialização(ões) da(s) variável(is) de controle
- ✓ Condição(ões)
- ✓ Atualização da(s) variável(is) de controle



# Encapsulamento de Dados

---

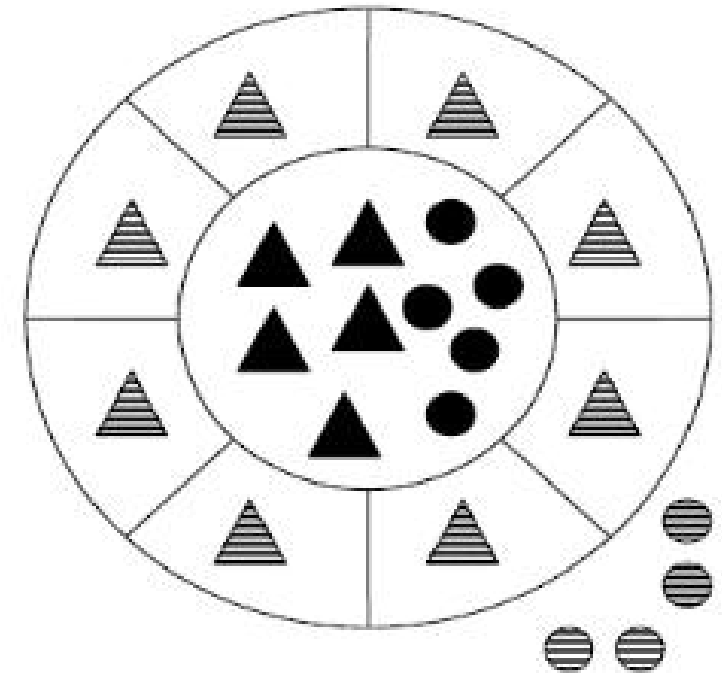
- ✔ O encapsulamento é as vezes referido como **ocultamento de informações**. Os usuários dos objetos não conhecem sua constituição e os utilizam através dos métodos públicos.
- ✔ O encapsulamento elimina dependências diretas na implementação, possibilitando a mudanças sem afetar outros sistemas que utilizem o objeto, desde que as assinaturas dos métodos não sejam alteradas.



# Encapsulamento de Dados

public      Menos restrito  
↓  
private    Mais restrito

- CLASSE**
- ▲ métodos públicos
  - ▲ métodos privados
  - dados privados
  - dados públicos (não recomendável)



# Encapsulamento de Dados

---

- ✓ Podemos bloquear o acesso aos atributos da classe.
- ✓ Criamos métodos (*set* e *get*) para acessar os atributos, já que eles não são mais acessíveis diretamente.
- ✓ Nesses métodos, podemos "validar" o que estão tentando armazenar nos atributos.

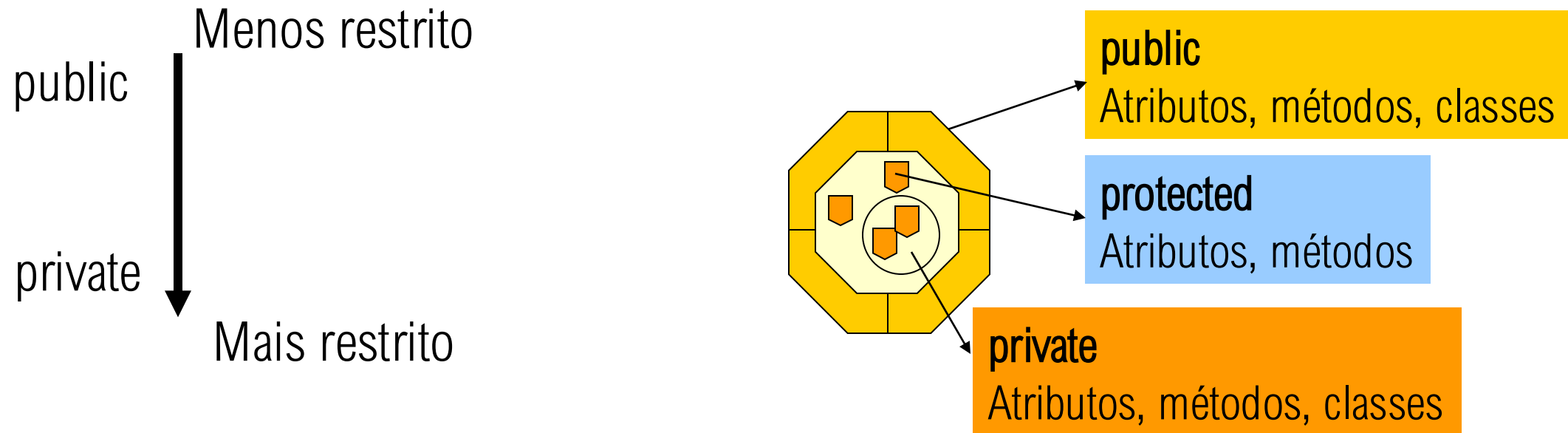


**SEGURANÇA**



# Encapsulamento de Dados

É uma proteção adicional dos dados do objeto de possíveis modificações impróprias, forçando o acesso a um nível mais baixo para tratamento de dados.



## Exemplo:

*Quando temos um arquivo protegido por senha de acesso, podemos dizer que ele está protegido, pois, apenas podemos lê-lo sem fazermos alteração*



# Modificadores de acesso

---

- ✓ Há três **modificadores** de acesso: **public**, **protected** e **private**;
- ✓ Atributos e Métodos podem ter os **três níveis** de acesso.
- ✓ Elementos **públicos** podem ser acessados diretamente por qualquer outra classe, utilizando um ponto (.) após o nome da variável.
- ✓ Elementos **privados** e **protegidos** não podem ser acessados diretamente utilizando o ponto.



# Modificadores de acesso

---

- ✓ **Private:** O modificador de acesso **private** é o mais restritivo de todos, variáveis e métodos com esse modificador são visíveis somente dentro da definição da própria classe, acessando-o diretamente ou através de uma instância da mesma classe.
- ✓ **Protected:** O modificador de acesso **protected** define que variáveis e métodos com esse modificador podem somente ser acessados por subclasses.
- ✓ **Public:** O mais abrangente de todos os tipos de acesso, o modificador **public** declara que elementos com esse modificador são acessíveis de qualquer classe Java.
- ✓ **Default:** Define que variáveis ou métodos podem somente ser acessados por classes do mesmo pacote.



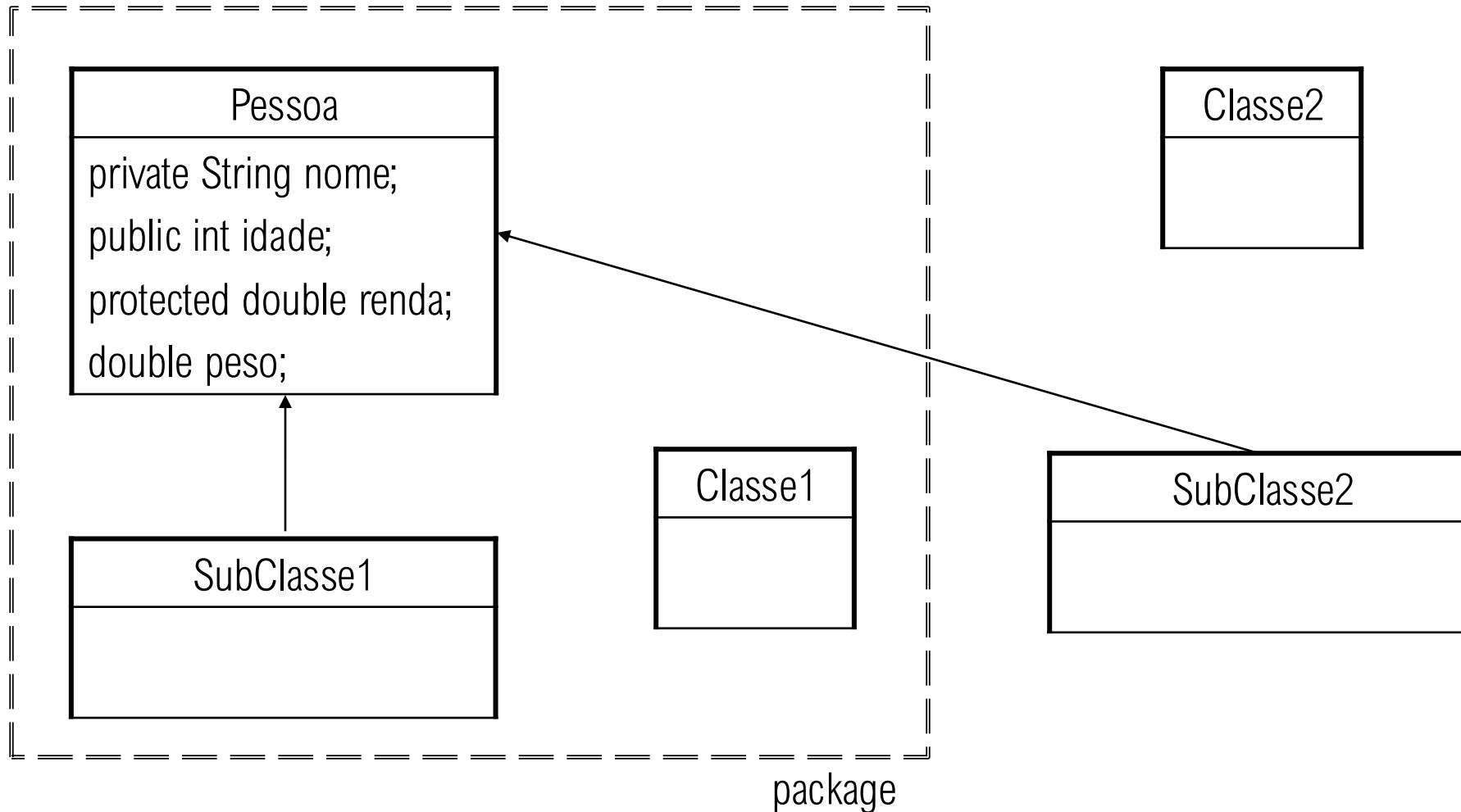
# Modificadores de acesso

|           |                                                                                                                                                                                                             |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| public    | ✓ Pode ser acessado por qualquer classe                                                                                                                                                                     |
| protected | <ul style="list-style-type: none"><li>✓ Pode ser acessado por qualquer sub-classe</li><li>✓ Pode ser acessado por qualquer classe do mesmo pacote</li><li>✓ Pode ser acessado pela própria classe</li></ul> |
| default   | <ul style="list-style-type: none"><li>✓ Pode ser acessado por qualquer classe do mesmo pacote</li><li>✓ Pode ser acessado pela própria classe</li></ul>                                                     |
| private   | ✓ Pode ser acessado somente pela própria classe                                                                                                                                                             |



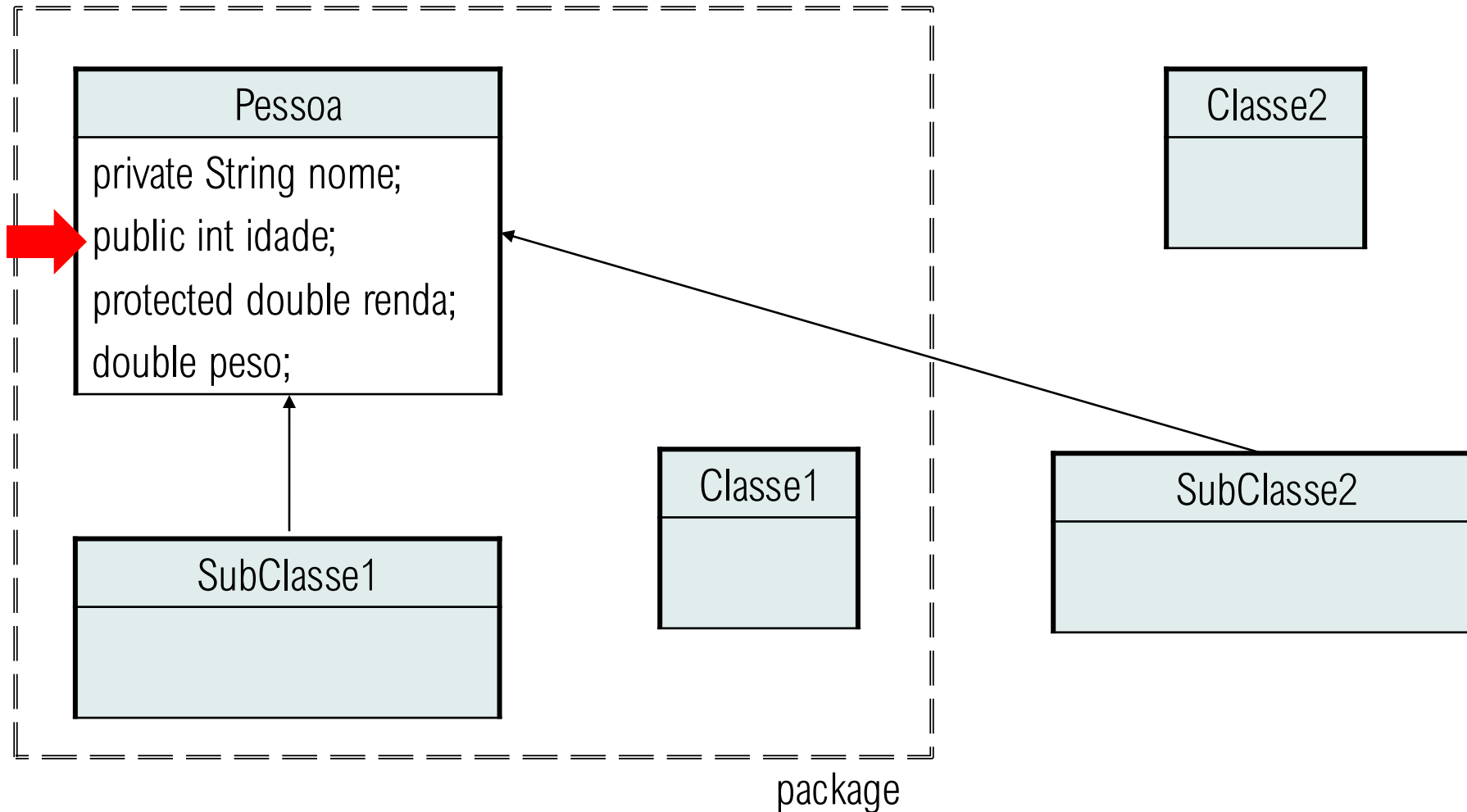
# Modificadores de acesso

## Visibilidade de variáveis e método: **Exemplo**



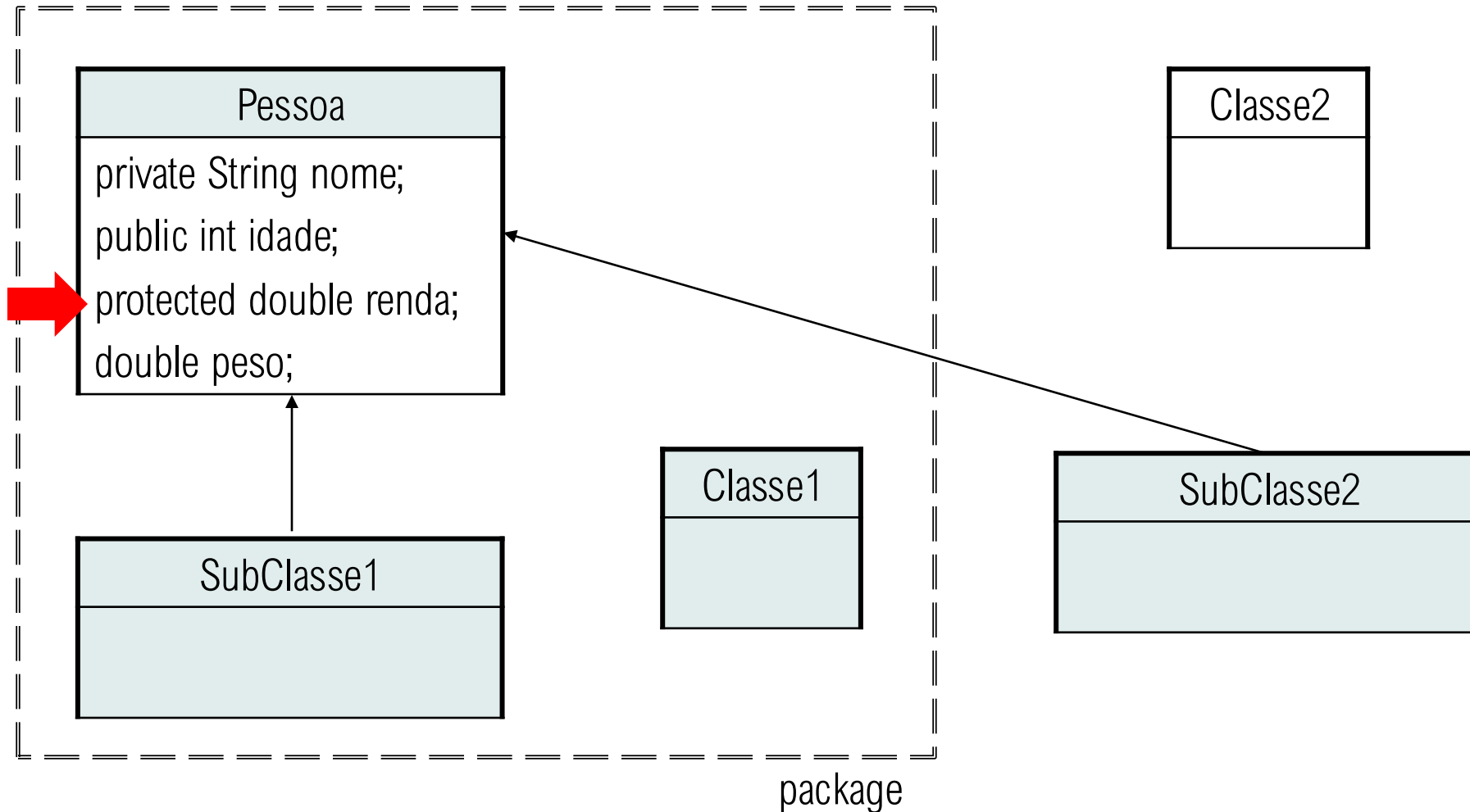
# Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **public**.



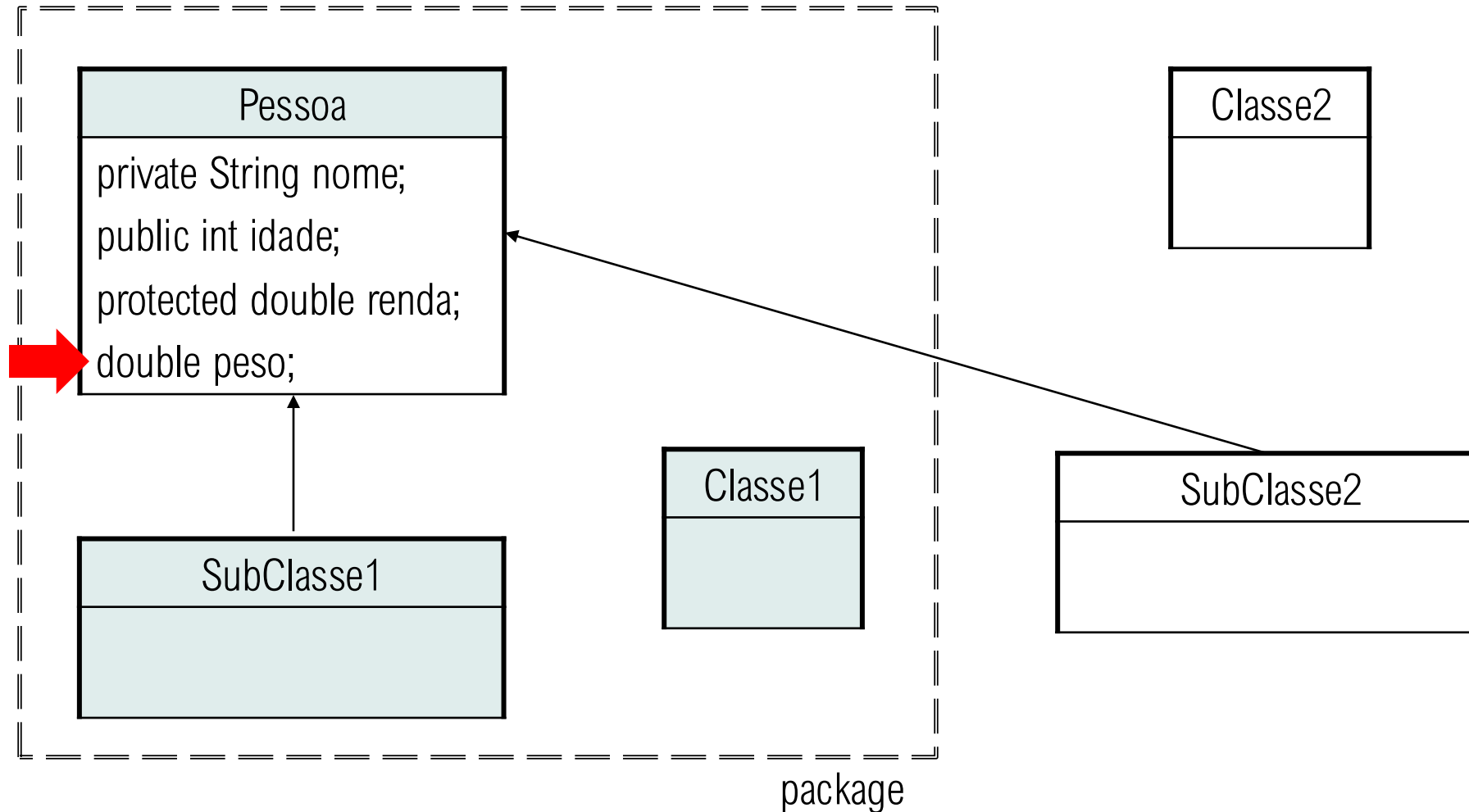
# Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **protected**.



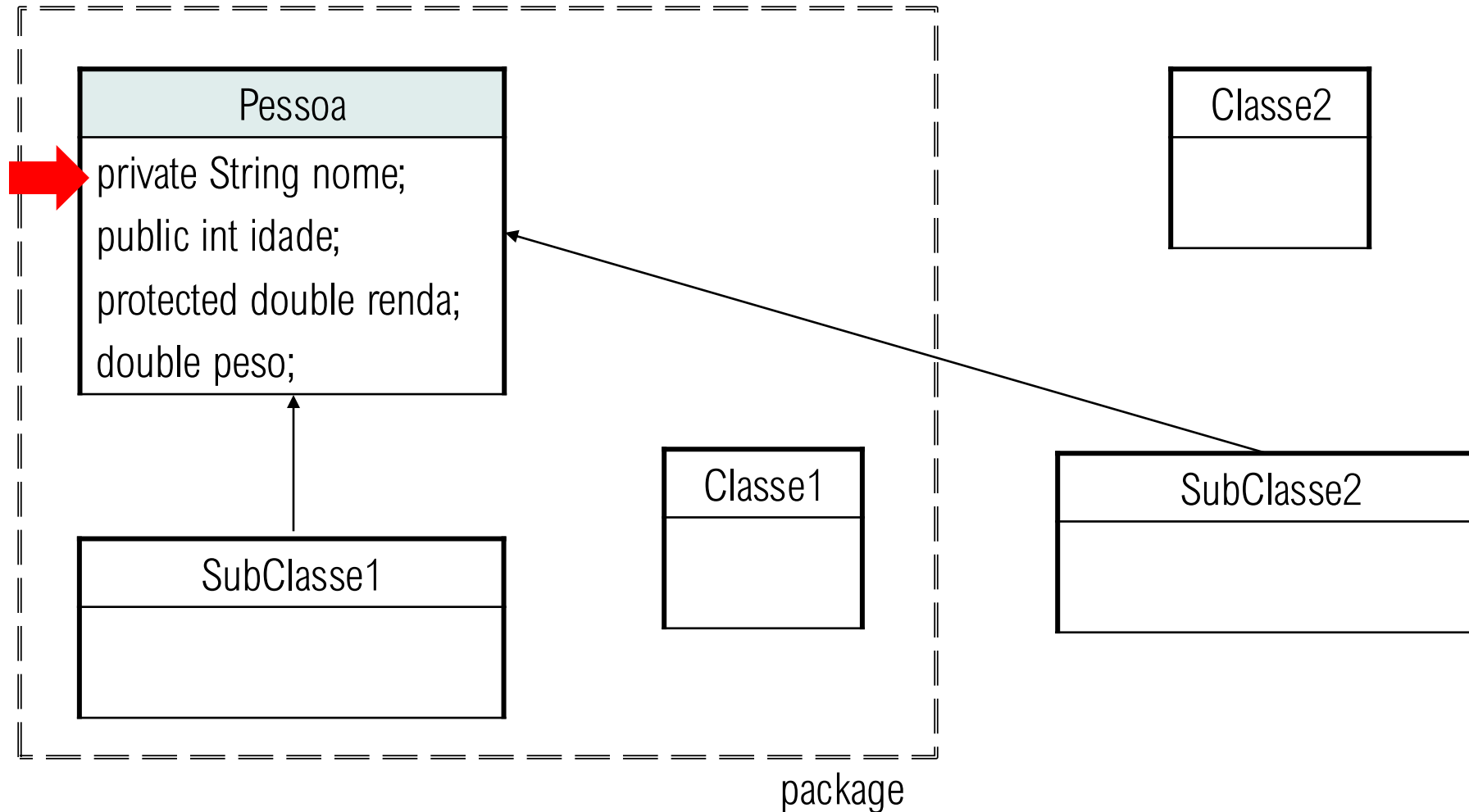
# Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **default**.



# Modificadores de acesso

Visibilidade de variáveis e métodos com o modificador **private**.





# Exemplo

| Pessoa               |
|----------------------|
| - nome: String       |
| + idade: int         |
| # renda: double      |
| imprimeDados(): void |

## Legenda:

- ( - ) indica **private**
- ( # ) indica **protected**
- ( + ) indica **public**

```
import javax.swing.JOptionPane;

public class Pessoa {
    //Atributos
    private String nome;
    public int idade;
    protected double renda;

    //métodos
    public void imprimeDados() {
        JOptionPane.showMessageDialog(null, "Nome: " + nome
            + "\nIdade: " + idade
            + "\nRenda: " + renda);
    }
}
```

**Observação:** a ausência de sinal na frente do atributo indica que ele é **default**

# Exemplo

```
public class TestePessoa {  
    public static void main(String[] args) {  
        Pessoa p1 = new Pessoa();  
  
        p1.nome = "Camila"; //erro de acesso  
        p1.idade = 30; //acesso bem sucedido  
        p1.renda = 1700.55; // acesso bem sucedido  
  
        p1.imprimeDados(); // acesso bem sucedido  
    }  
}
```



# Métodos de acesso

---

- ✓ Servem como métodos de leitura/escrita aos atributos de classes
- ✓ Um método de leitura para um atributo deve ser chamado de **getXxx** (onde Xxx é o nome do atributo). Este método não recebe nada como parâmetro, e retorna o mesmo tipo do atributo.
- ✓ Já um método de gravação deve ser chamado **setXxx**, não retorna nada (geralmente), e recebe como parâmetro o valor que deve ser armazenado no atributo.



# Exemplo

| Pessoa                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- nome: String</li><li>- idade: int</li><li>- renda: double</li></ul>                                                                                                            |
| Pessoa()<br>Pessoa(n: String, i: int, r: double)<br>getNome(): String<br>getIdade(): int<br>getRenda(): double<br>setNome(String n): void<br>setIdade(int i): void<br>setRenda(double r): void<br>imprimeDados(): void |

```
...
//Métodos de acesso
public String getNome() {
    return nome;
}
public int getIdade() {
    return idade;
}
public double getRenda() {
    return renda;
}
public void setNome(String n) {
    nome = n;
}
public void setIdade(int i) {
    idade = i;
}
public void setRenda(double r) {
    renda = r;
}
...
```

# Herança

---

- ✓ Como o próprio nome sugere, na orientação a objetos o termo herança **se refere a algo herdado**.
- ✓ Em Java, a herança ocorre quando uma **classe** passa a herdar **características** (atributos e métodos) **definidas em uma outra classe**, especificada como sendo sua ancestral ou superclasse.
- ✓ A técnica da herança **possibilita** o **compartilhamento** ou **reaproveitamento** de recursos definidos anteriormente em uma outra classe.
- ✓ A classe **fornecedora** dos recursos recebe o nome de **superclasse** e a **receptora** dos recursos de **subclasse**.



# Herança

Uma classe derivada **herda a estrutura de atributos** e métodos de sua classe “base”, mas pode seletivamente:

- ✓ adicionar novos métodos
- ✓ estender a estrutura de dados
- ✓ redefinir a implementação de métodos já existentes

Exemplo:

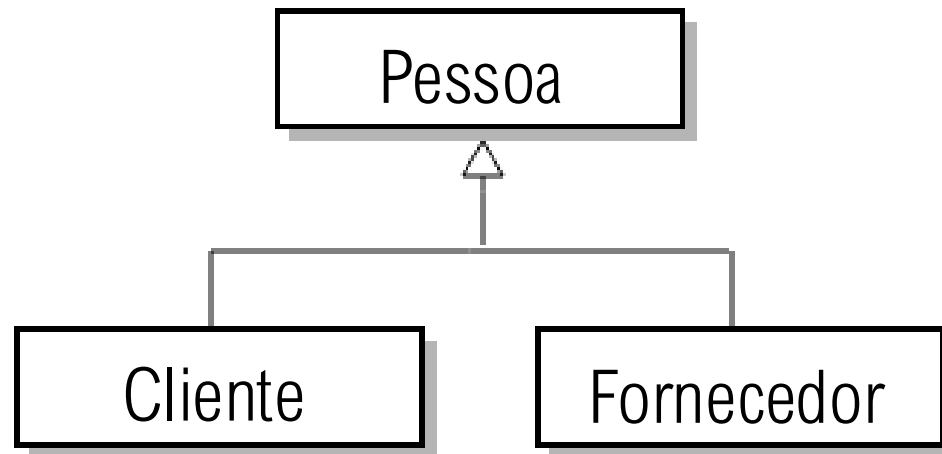
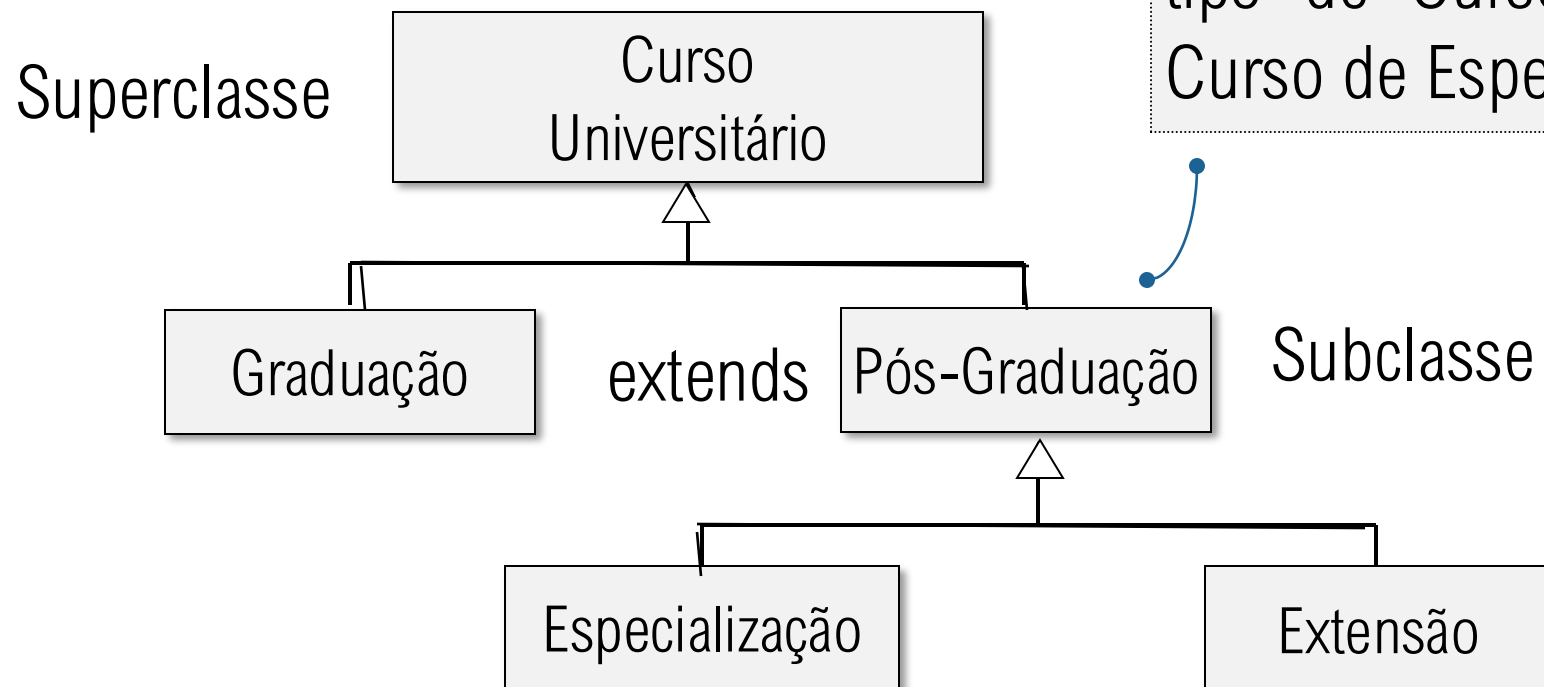


Diagrama UML simplificado  
(não mostra os métodos e atributos)

# Herança



Podemos dizer que Pós-Graduação é um tipo de Curso Universitário, assim como Curso de Especialização ou de Extensão.



# Herança em Java

---

Para dizer que uma classe implementa herança, utilizamos a palavra-chave ***extends*** logo após a declaração da mesma:

```
public class Filho extends Pai{ }
```

Se uma classe não contém a declaração *extends*, automaticamente “herda” as características da classe ***Object***

Alguns métodos da classe ***Object*** são:

- ✓ `toString( ): String`
- ✓ `equals(Object): boolean`



# Herança – Modificadores de Acesso

- ✓ **protected**: podem ser acessados pelos membros da subclasse
- ✓ **private**: só podem ser acessados pela própria classe

```
public class Pai {  
  
    public int a;  
    protected int b;  
    private int c;  
  
    public Pai() {  
        a = 10;  
        b = 24;  
        c = 66;  
    }  
}
```

Pai.java

```
public class Filho extends Pai {  
  
    public void imprimeFilho() {  
        System.out.println("Valor de a: " + a);  
        System.out.println("Valor de b: " + b);  
        System.out.println("Valor de c: " + c);  
    }  
}
```

Filho.java

```
public class TestaFilho {  
  
    public static void main(String[] args) {  
        Filho f = new Filho();  
        f.imprimeFilho();  
    }  
}
```

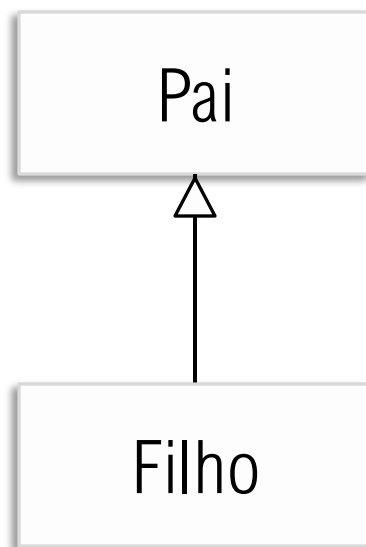
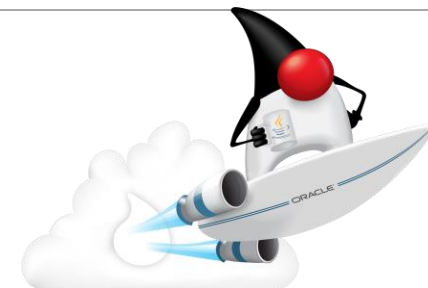
TestaFilho.java

Isso dá  
erro!!!!



# Herança – Modificadores de Acesso

- ✓ **protected:** podem ser acessados pelos membros da subclasse
- ✓ **private:** só podem ser acessados pela própria classe



- ✓ A subclasse Filho **herda** todos os atributos e métodos da superclasse Pai.
- ✓ Ela também herda os atributos e métodos privados, porém, **não consegue acessá-los diretamente.**
- ✓ Para acessar um membro privado na subclasse indiretamente, seria necessário que superclasse expusesse de forma visível esse atributo ou método privado.

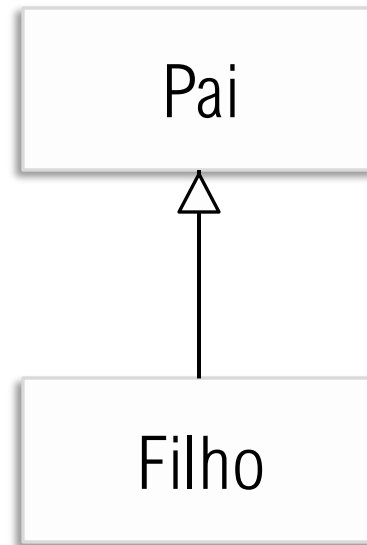
# Herança – Modificadores de Acesso

- ✓ **protected:** podem ser acessados pelos membros da subclasse
- ✓ **private:** só podem ser acessados pela própria classe



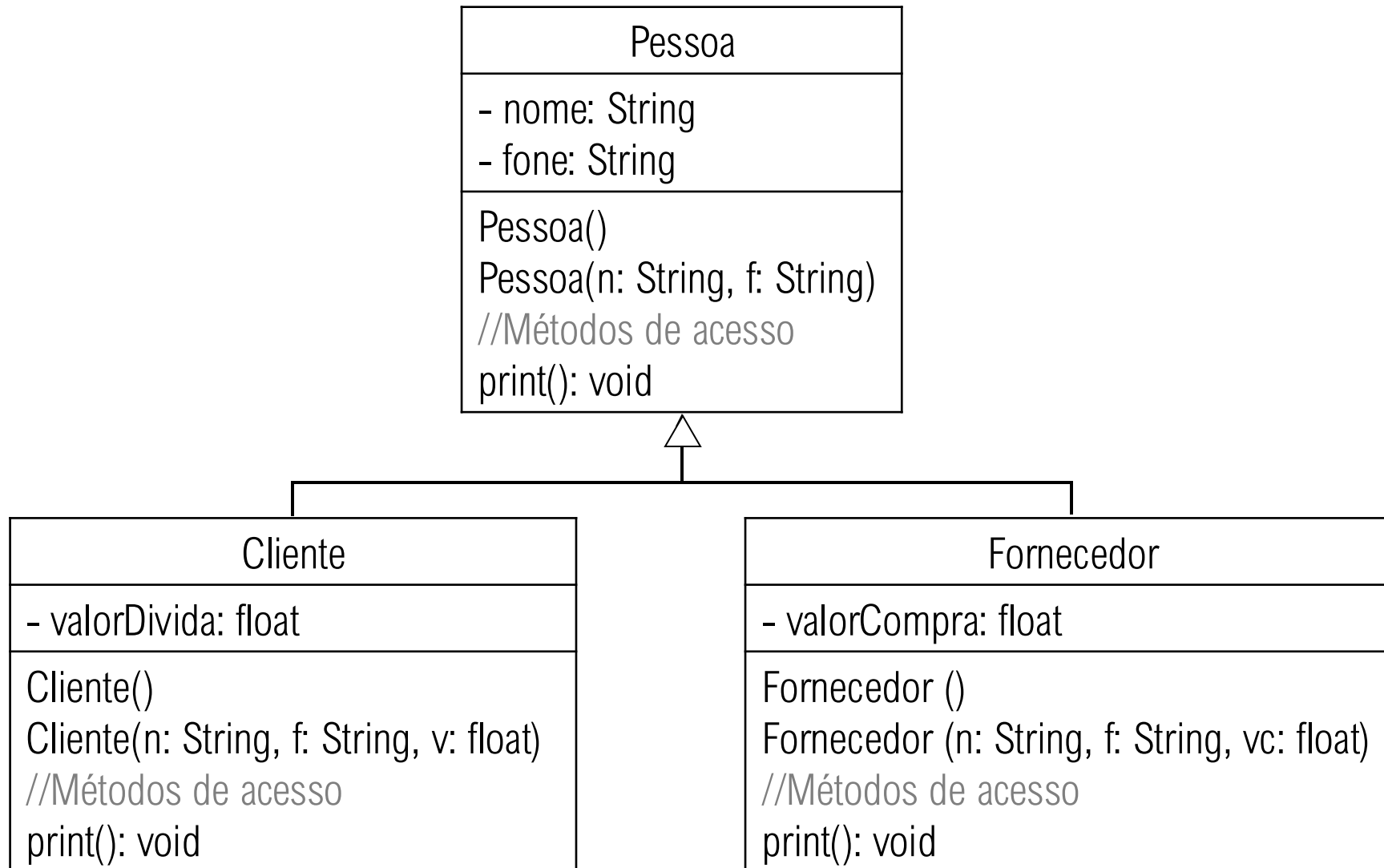
## Super e Sub classe:

A nomenclatura mais encontrada é que Pai é a **superclasse** de Filho. Desta forma, Filho é uma **subclasse** de Pai. Pode-se dizer também que todo Filho **é um** Pai.



# Herança: Exemplos de aplicação

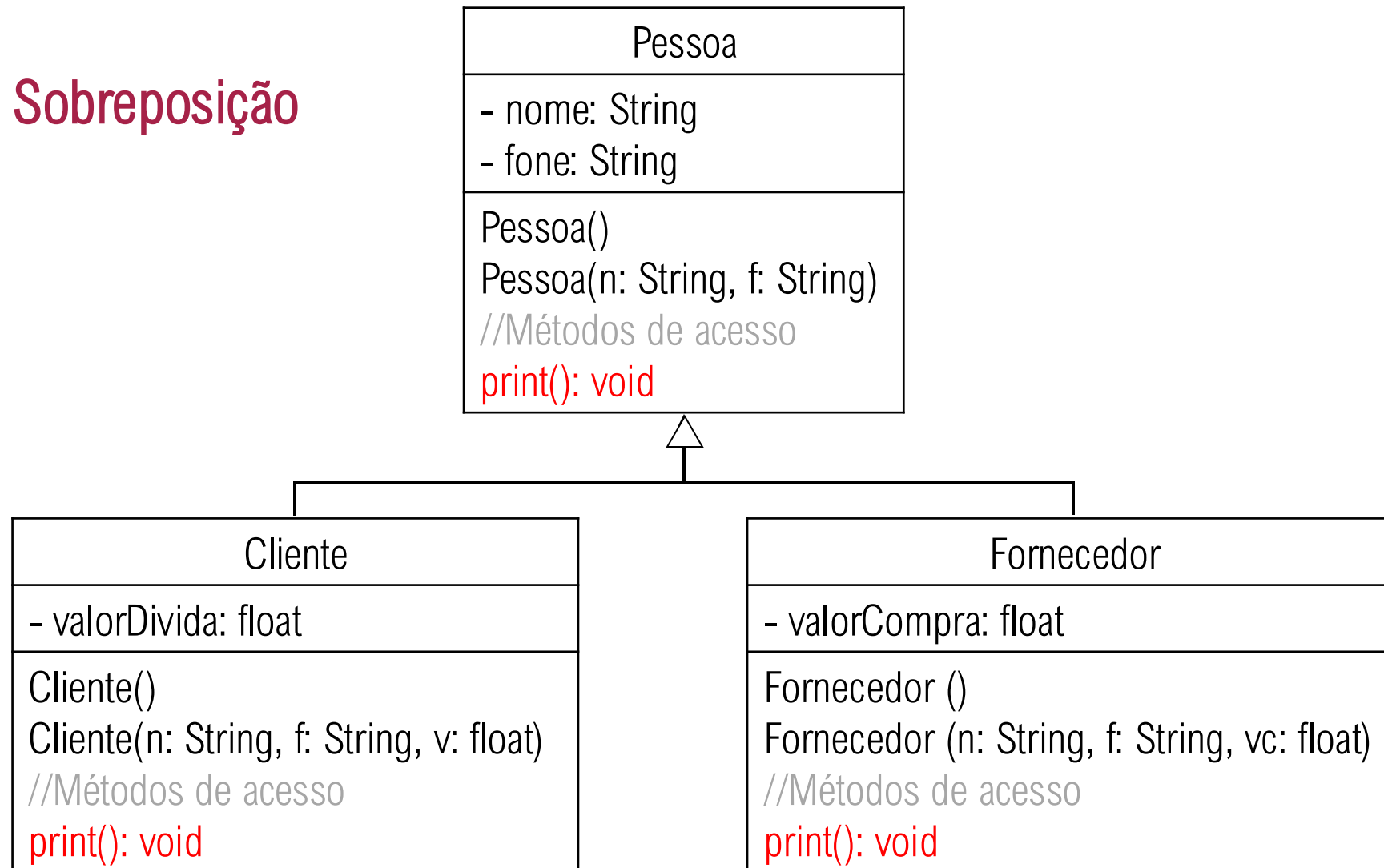
1



# Sobreposição (Overriding) e Extensão

1

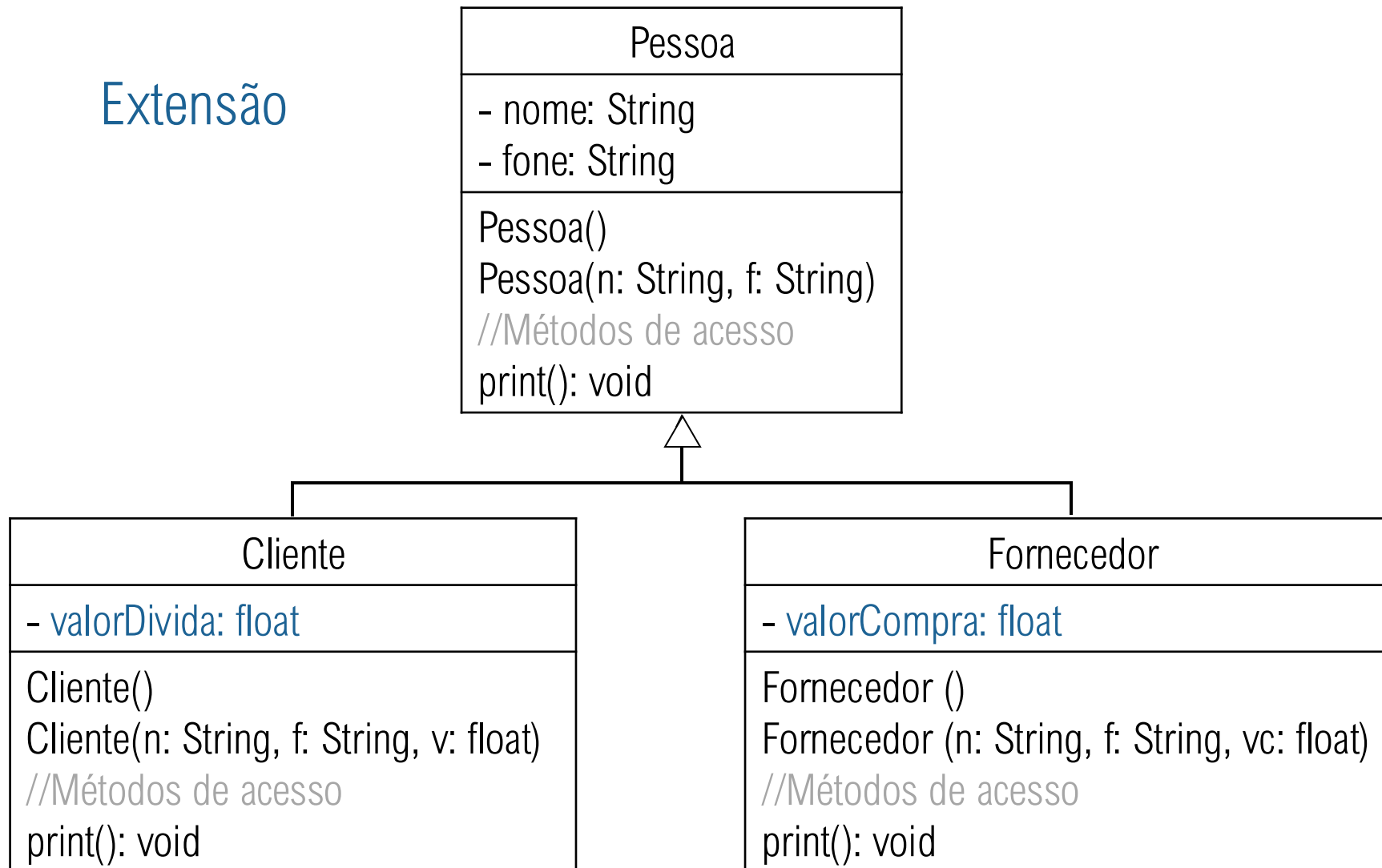
## Sobreposição



# Sobreposição (Overriding) e Extensão

1

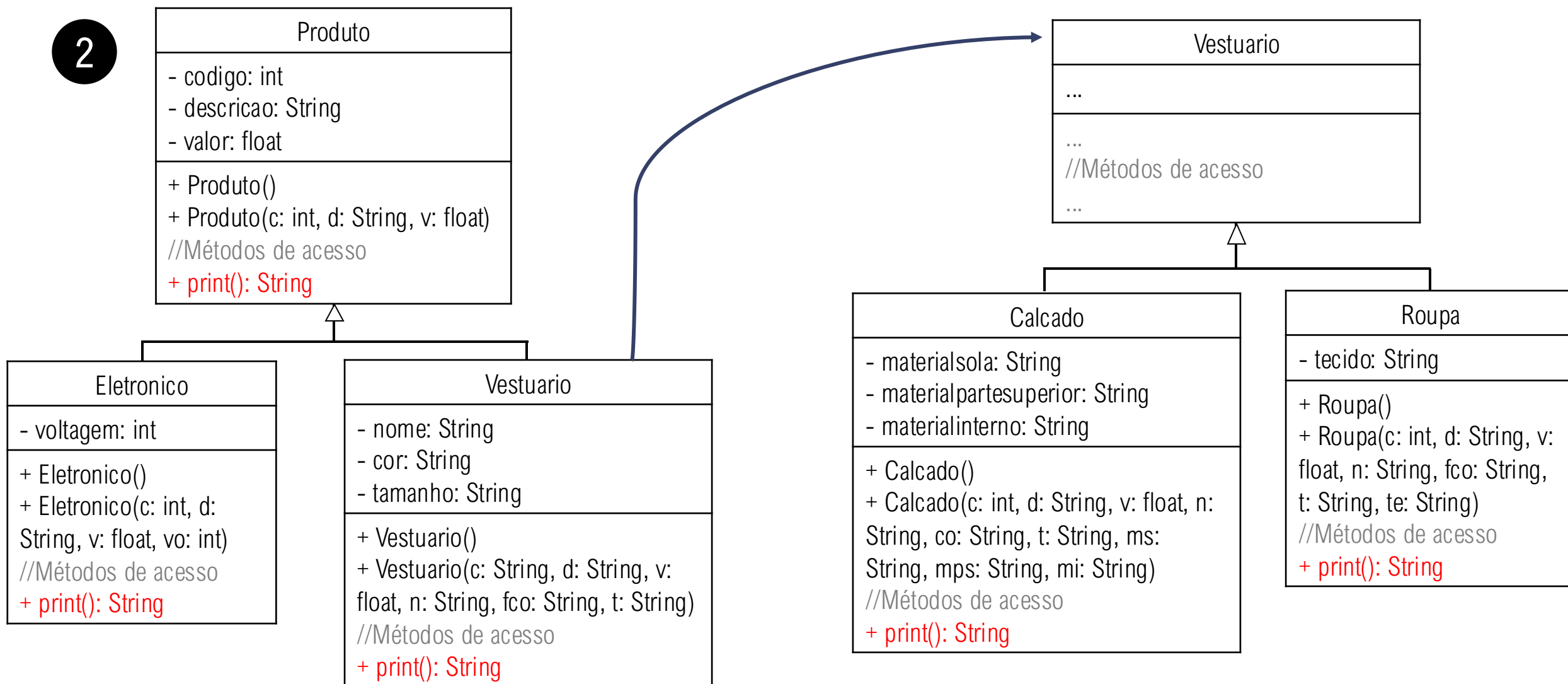
## Extensão





# Herança: Exemplos de aplicação

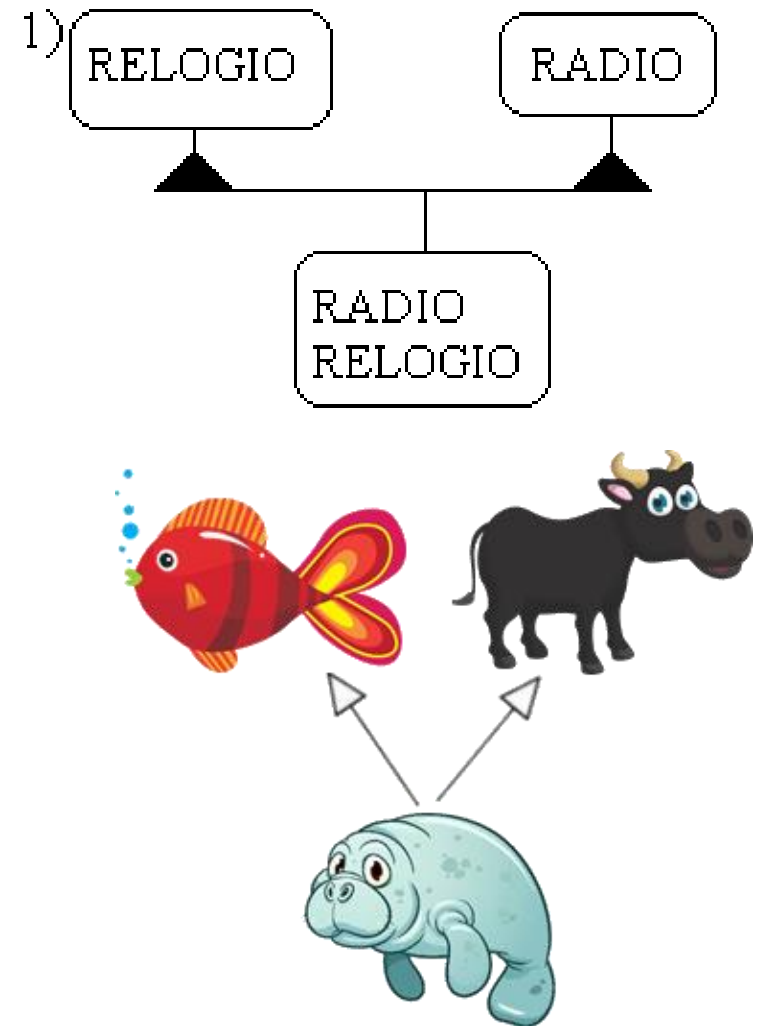
2





# Herança múltipla

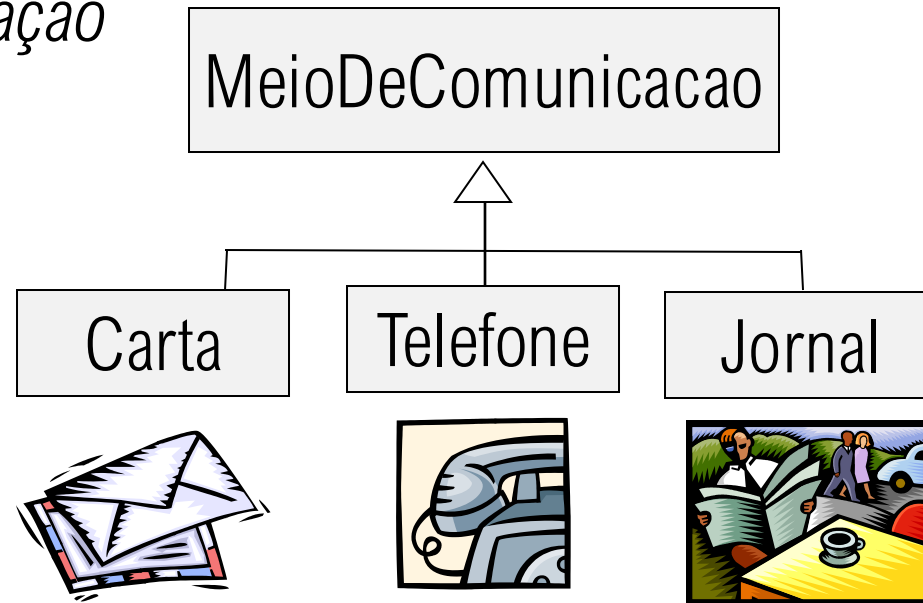
- ✓ Algumas linguagens OO permitem fazer herança múltipla como o C++, o que significa que uma subclasse pode herdar características de duas ou mais classes.
- ✓ Isso permite que uma classe agrupe atributos e métodos de várias classes.
- ✓ Java não tem herança múltipla



# Abstração de Dados: Classes Abstratas

Nos ajuda a lidar com a complexidade.

*Generalização*



*Especialização*

A classe *MeiodeComunicacao* neste caso é abstrata e pode representar um domínio.



# Classes Abstratas

Uma classe abstrata é uma classe que:

- ✓ Provê organização
- ✓ **Não possui “instâncias”**
- ✓ Pode ter métodos abstratos ou concretos

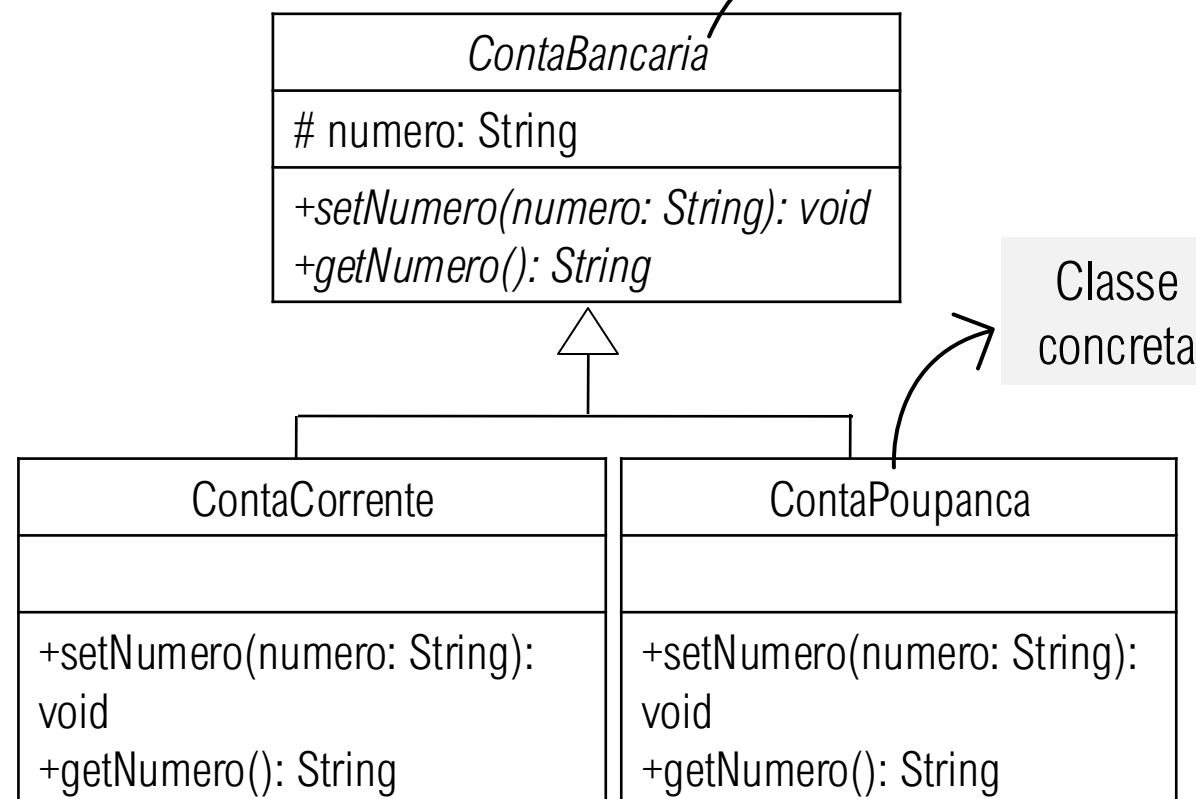
Se você definir um método abstrato sua classe OBRIGATORIAMENTE **DEVE SER ABSTRATA.**



No diagrama o nome da classe e métodos abstratos são apresentados em itálico

Você pode ter métodos concretos em uma classe abstrata

Classe abstrata

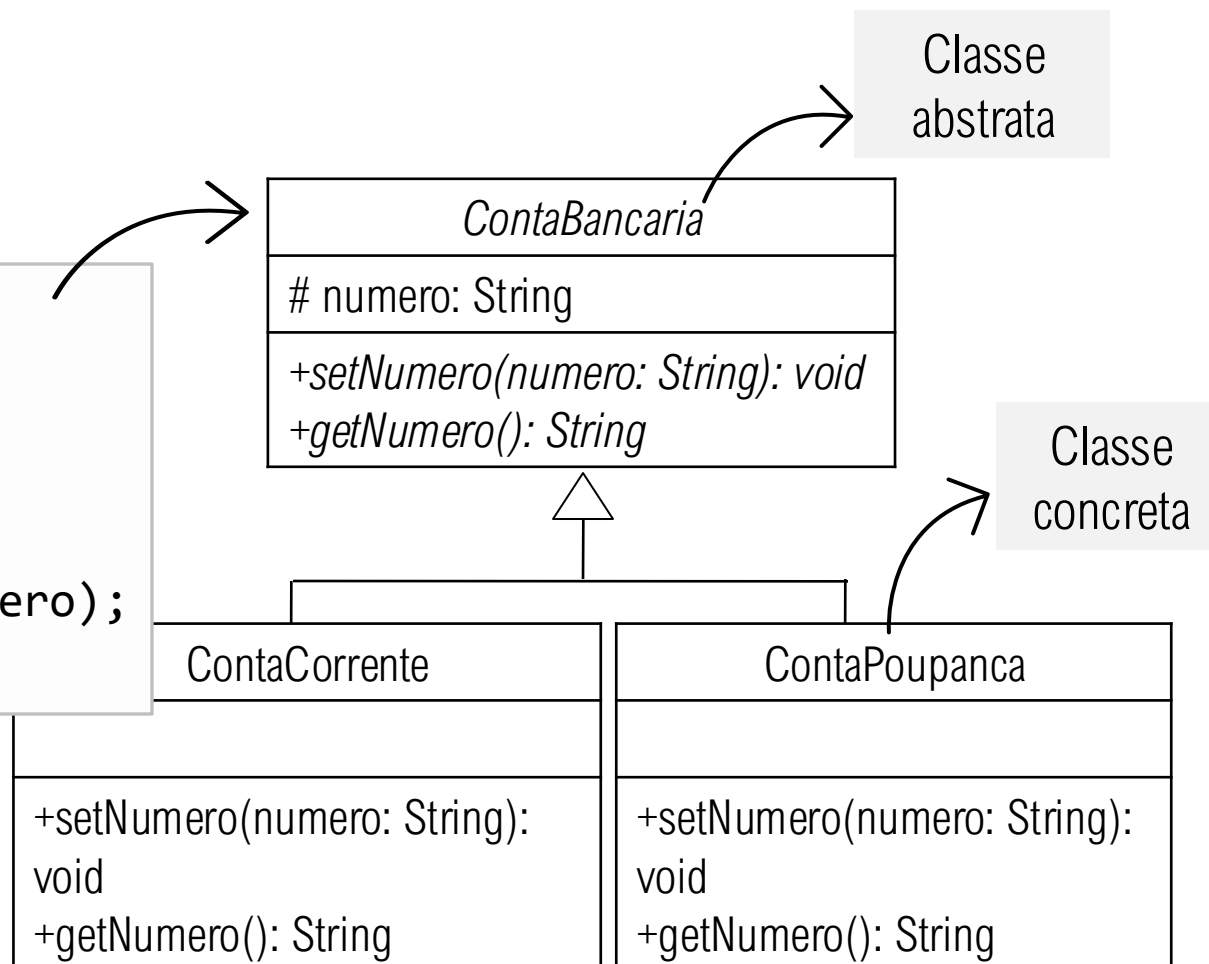


# Classes Abstratas

Uma classe abstrata é uma classe que:

- ✓ Provê organização
- ✓ **Não possui “instâncias”**
- ✓ Pode ter métodos abstratos ou concretos

```
public abstract class ContaBancaria {  
    protected String numero;  
    public ContaBancaria() { }  
    public abstract int getNumero();  
    public abstract void setNumero(String numero);  
}
```



No diagrama o nome da classe e métodos abstratos são apresentados em itálico

# Classes Abstratas

- ✓ São criadas quando não se pretende criar objetos a partir delas
- ✓ São normalmente usadas como superclasses
- ✓ Se caracterizam por serem muito genéricas
- ✓ A assinatura da classe abstrata é da seguinte forma:

```
public abstract class NomeDaClasse{
```

```
...  
}
```

Palavra-chave que identifica a classe  
como abstrata



# Métodos Abstratos

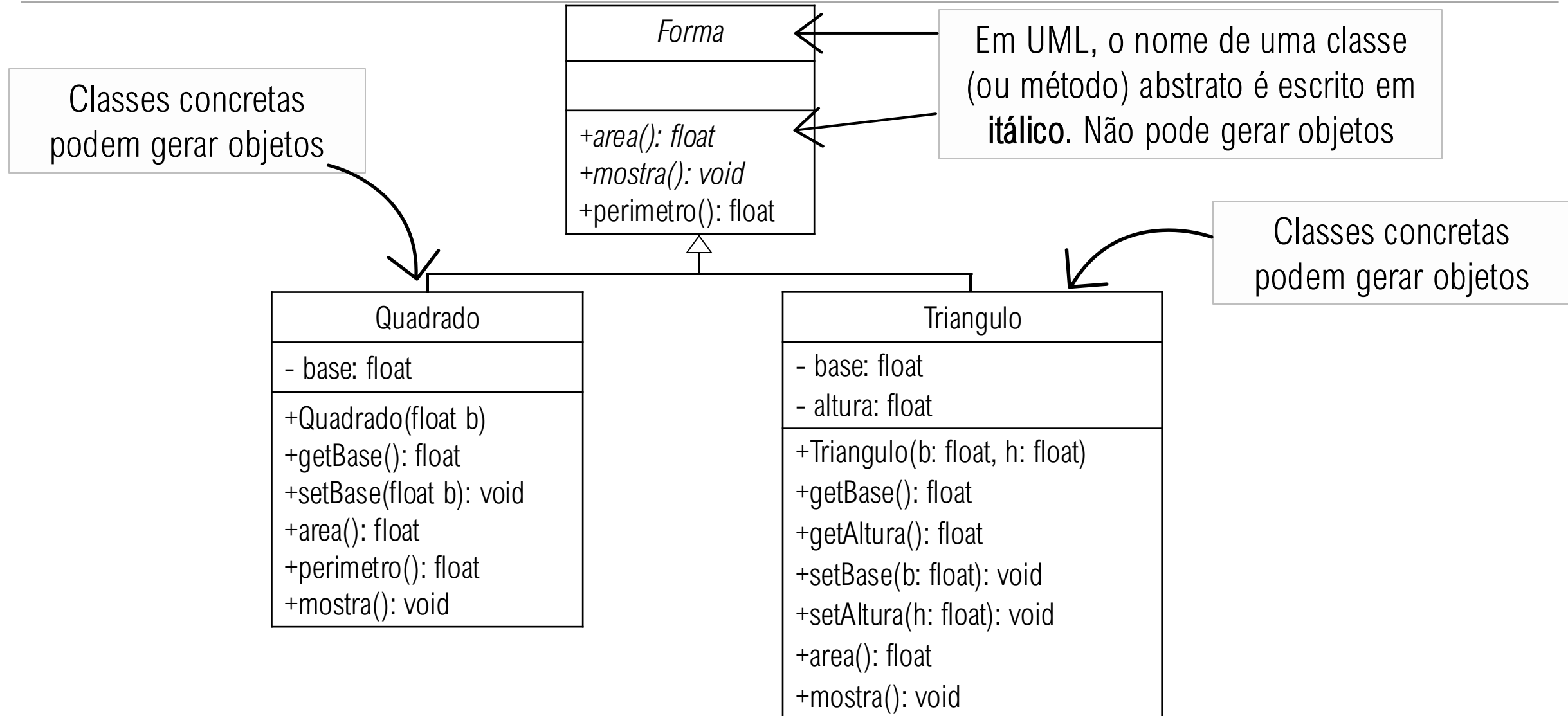
- ✔ Os métodos abstratos definidos em uma classe abstrata devem obrigatoriamente implementados em uma classe concreta. Mas se uma classe abstrata herdar outra classe abstrata, a classe que herda não precisa implementar os métodos abstratos.
- ✔ Indica que todas as classes filhas concretas devem reescrever esse método e ele deve ser um método concreto nas classes filhas;
- ✔ Os métodos que são abstratos têm um comportamento diferente nas classes filhas, por isso não possuem corpo;
- ✔ A assinatura do método abstrato é da seguinte forma:

`public abstract tipoRetorno NomeDoMetodo();`

Palavra-chave que identifica o método como abstrato

Não é definida uma forma de implementação específica.

# Classes Abstratas



# Classes abstratas: Exemplos de aplicação

3

```
public abstract class Forma {  
    //Atributos  
  
    //Métodos  
    public abstract float area();  
    public abstract void mostra();  
    public float perimetro(){return 0f;}  
}
```

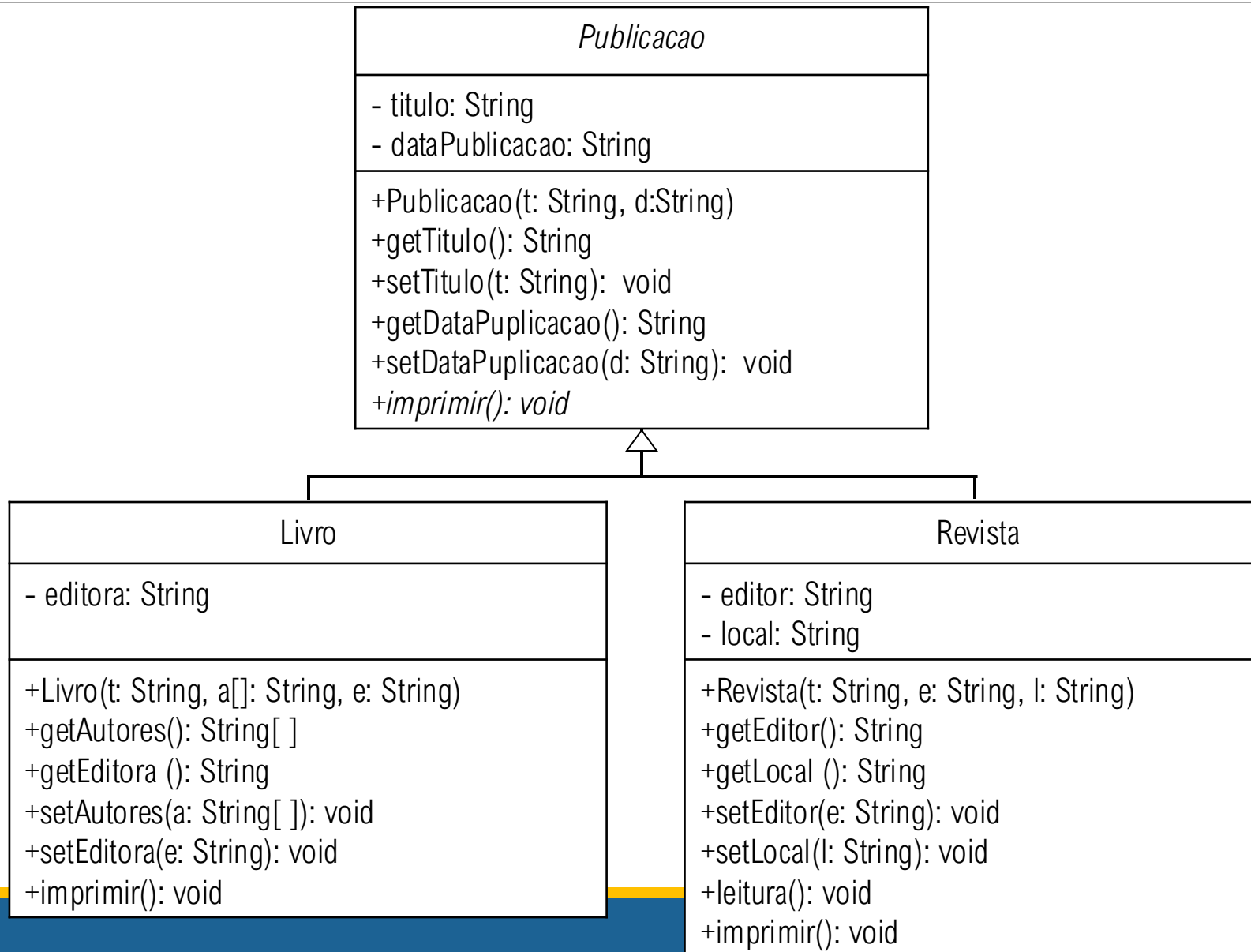
Métodos abstratos, não  
possuem implementação





# Classes abstratas: Exemplos de aplicação

4



# Interfaces

---

- ✔ São estruturas similares às classes abstratas, que definem a especificação de funcionalidades, sem a implementação das mesmas.
- ✔ Uma interface Java pode ser definida como uma “**classe abstrata pura**”, pois não pode possuir atributos (exceto constantes) nem definições de métodos, nem construtor.



# Interfaces

---

- ✔ Apenas métodos públicos podem ser declarados nelas, mas não podem ser definidos.
  - Todos os métodos declarados dentro de uma interface são, implicitamente, *public* e *abstract*
- ✔ Da mesma forma, não é possível definir atributos - apenas constantes públicas.
  - Todos os atributos declarados dentro de uma interface são, implicitamente, *public*, *static* e *final*
- ✔ São proibidas algumas declarações dentro das interfaces, dentre elas:
  - *private*, *protected*, *transient*, *volatile* e *synchronized*.

# Interfaces

---

- ✓ A palavra-chave *interface* é utilizada em vez de *class* no cabeçalho da declaração:

```
public interface nomeInterface { ... }
```

- ✓ Todos os métodos são abstratos. A palavra-chave *abstract* não é necessária



# Classes abstratas vs Interfaces

---

- ✓ A diferença entre uma classe abstrata e uma interface Java é que a **interface** obrigatoriamente não pode ter um “corpo” associado.
- ✓ Enquanto uma **classe abstrata** é “estendida” (palavra chave *extends*) por classes derivadas, uma **interface** Java é “implementada” (palavra chave *implements*) por outras classes.



# Classes abstratas vs Interfaces

---

```
public class nomeClasse implements nomeInterface
```

OU

```
public class nomeClasse implements nomeInterface1, nomeInterface2
```

OU

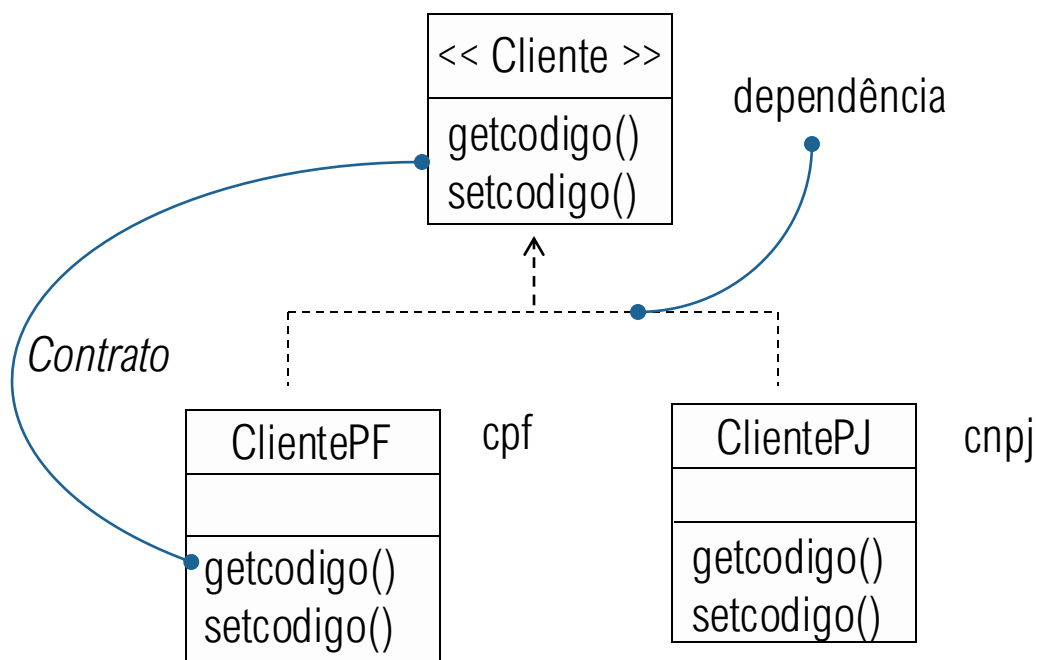
```
public class nomeClasse extends nomeHerança implements nomeInterface
```

A interface **não faz parte da hierarquia** de classes em Java.

# Implementação de Interfaces

Uma interface estabelece uma espécie de contrato que é obedecido por uma classe.

- ✔ Quando uma classe implementa uma interface, garante-se que todas as funcionalidades especificadas pela interface serão oferecidas pela classe.



- ✔ Uma **classe pode implementar várias interfaces**, sendo um mecanismo elegante para se trabalhar com herança múltipla em Java.

# Interfaces - vantagens

---

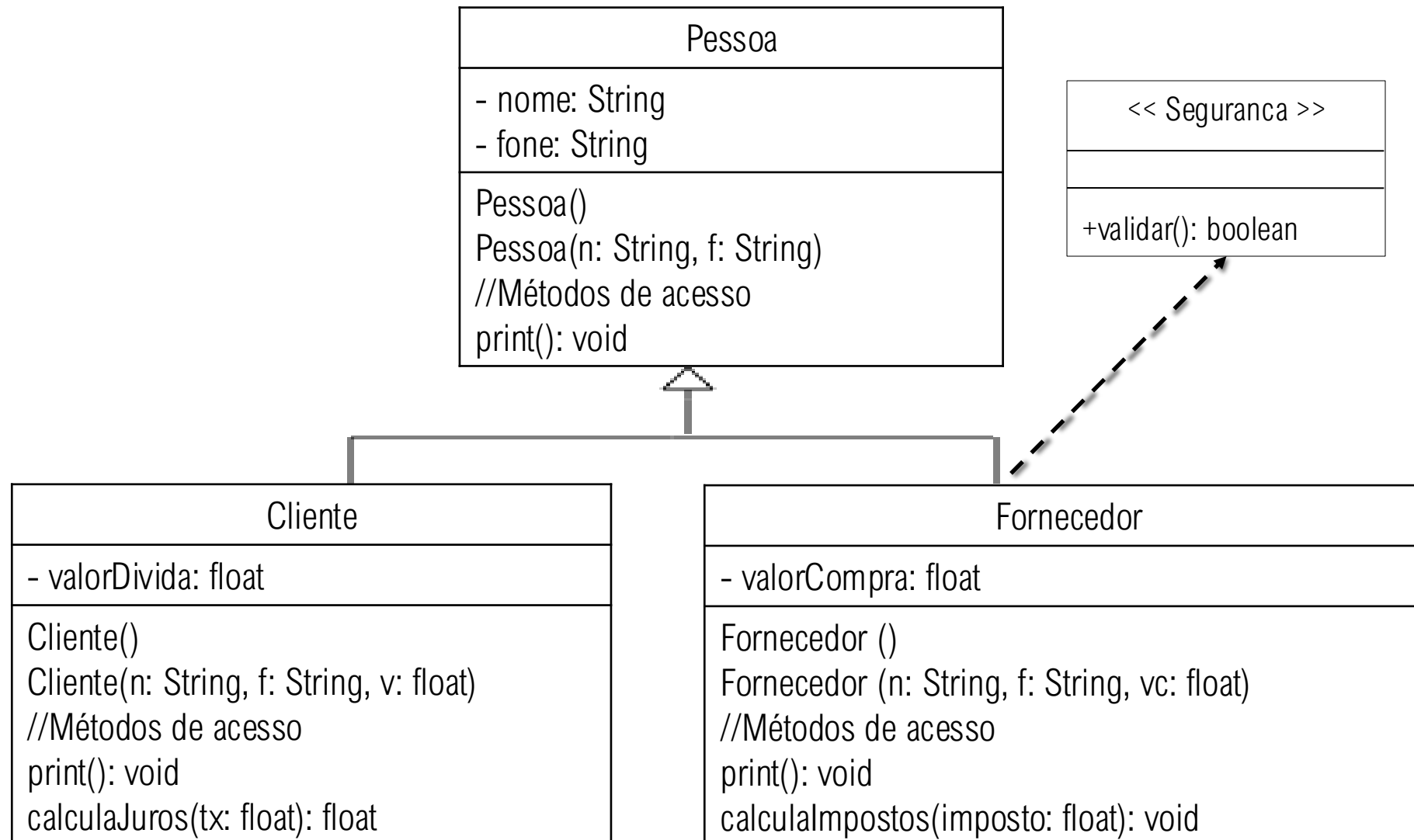
- ✓ Implementar similaridades entre classes não-relacionadas, sem forçar relacionamentos na hierarquia de classes.
- ✓ Definir métodos que uma ou mais classes devam implementar.
- ✓ Revelar apenas as interfaces de um objeto, sem revelar a estrutura de sua classe, ajuda assim esconder a complexidade da arquitetura de componentes;
- ✓ Oferece uma forma simplificada de implementar herança múltipla.
- ✓ Facilitar o entendimento do código – separação entre o que a classe “é” do que ela “faz”.





# Interfaces: Exemplos de aplicação

5



# Interfaces: Exemplos de aplicação

6

```
public interface Imprimivel {  
    //constante  
    final char nlin='\n';  
  
    //métodos  
    public String formatoString();  
    public void formatoSystemOut();  
}
```

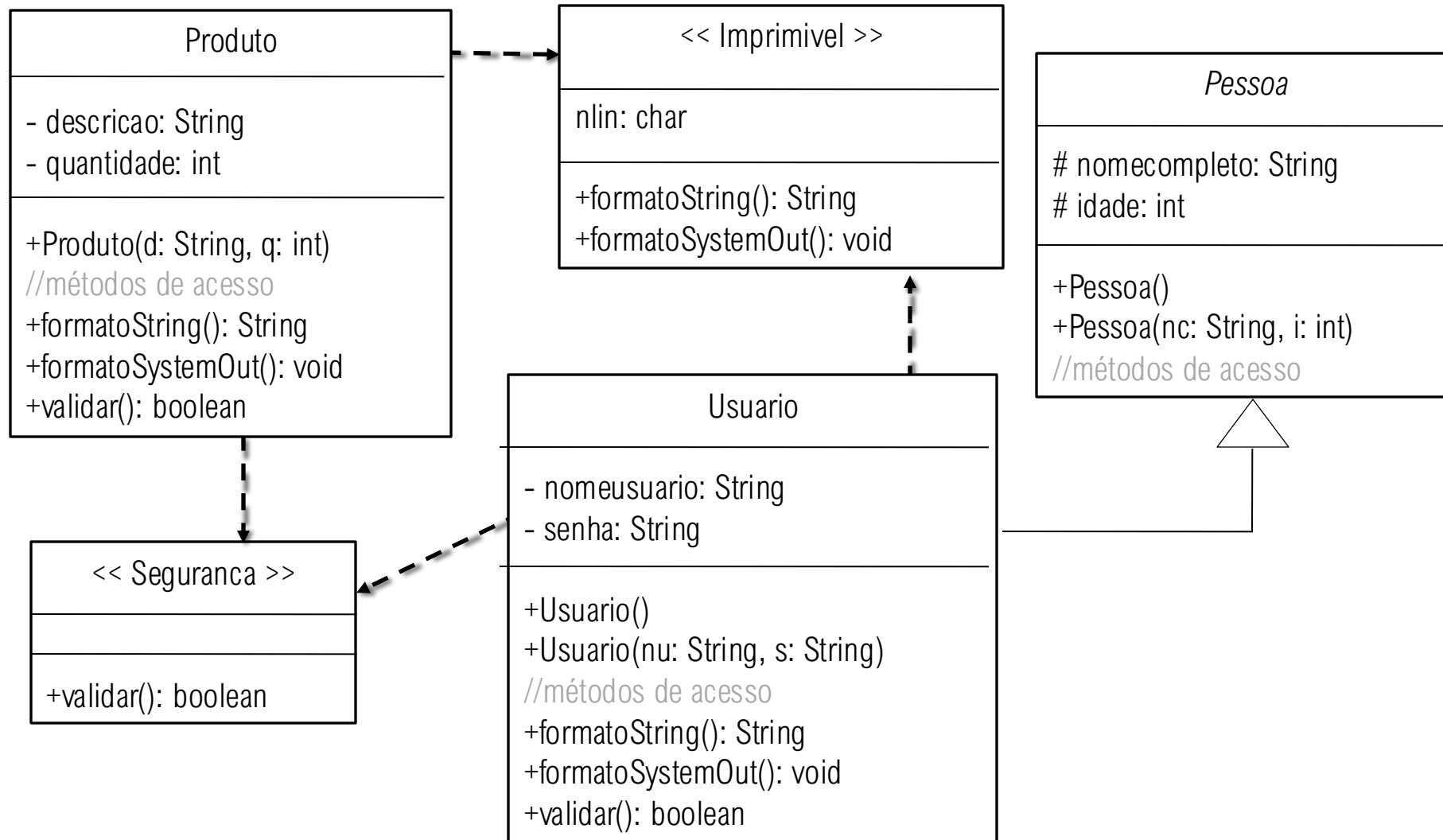
|                                                       |
|-------------------------------------------------------|
| << Imprimivel >>                                      |
| nlin: char                                            |
| +formatoString(): String<br>+formatoSystemOut(): void |

```
public interface Seguranca {  
    public boolean validar();  
}
```

|                     |
|---------------------|
| << Seguranca >>     |
|                     |
| +validar(): boolean |

# Interfaces: Exemplos de aplicação

6



# Interfaces: Exemplos de aplicação

---

6

- ✓ Todas as classes que implementem a interface “Imprimivel” terão a “obrigação” de implementar os métodos `formatoString()` e `formatoSystemOut()`;
- ✓ Isso faz com que você tenha um padrão, e não importa como os métodos serão implementados, o usuário saberá que esses métodos existirão;
- ✓ Outras classes como `ContaCorrente`, `Alunos`, `FichaClinica`, etc, podem implementar essa interface e terão os métodos da interface também.

# Classes abstratas vs Interfaces

---



Use classes abstratas quando você quiser definir um “template” para subclasses e você possui alguma implementação (métodos concretos) que todas as subclasses podem utilizar.



Use interfaces quando você quiser definir uma regra que todas as classes que implementem a interface devem seguir, independentemente se pertencem a alguma hierarquia de classes ou não.



# Material preparado pelos professores

---

- ✔ Marco, Cristiane, Lédon da Universidade Cruzeiro do Sul

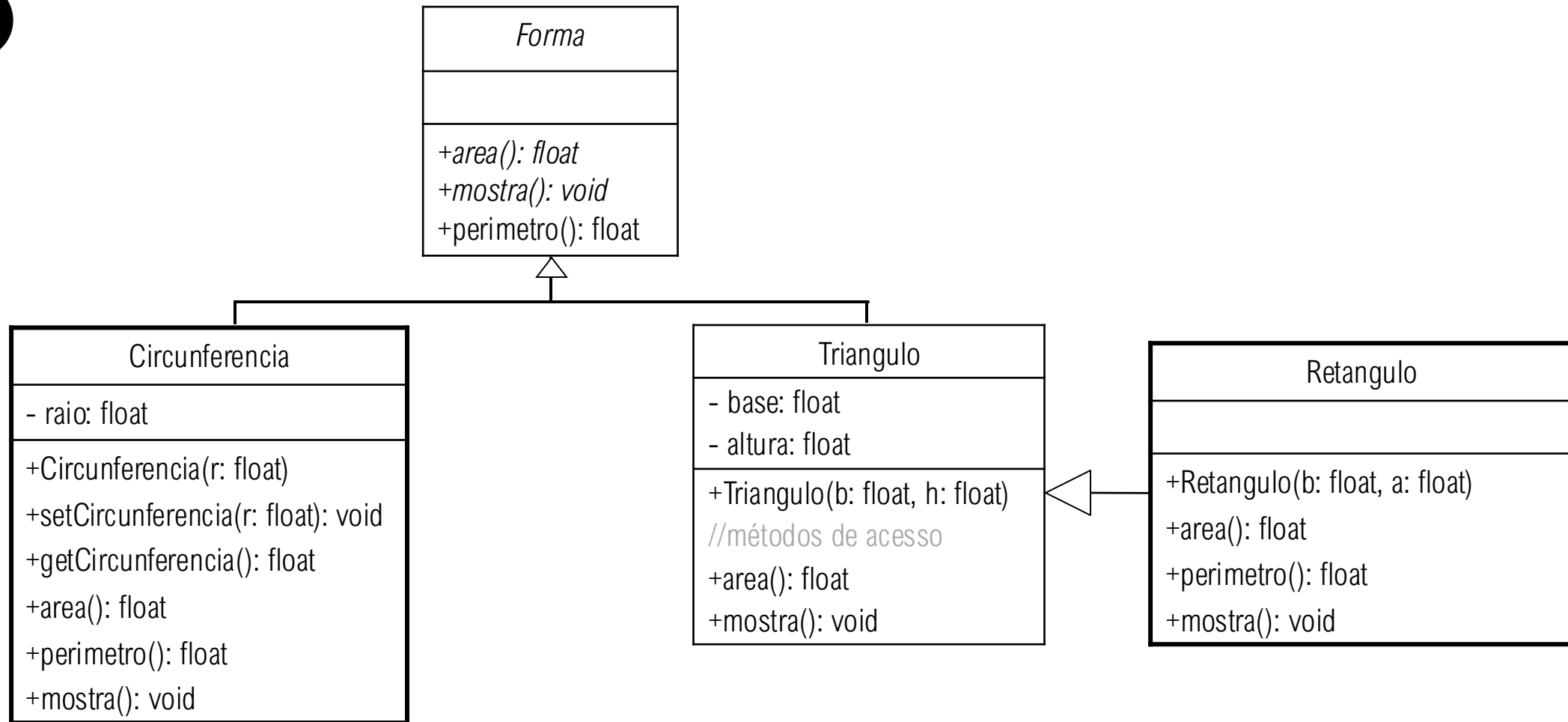
# Exercício para entregar

---



# Exercícios

2





# Exercícios

- 2 Crie a classe ao lado como subclasse de Forma:
- ✓ O método `area()` deve retornar valor da área da circunferência, sabendo que
$$\text{area} = \pi * r^2$$
  - ✓ O método `perimetro()` deve retornar o valor do perímetro:
$$\text{perimetro} = 2 * \pi * r$$
  - ✓ Em ambos os métodos utilize a constante `Math.PI` da classe `Math`.
  - ✓ O método `mostra` deve exibir os valores de todos os atributos da classe

| Circunferencia                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - raio: float                                                                                                                                              |
| +Circunferencia(r: float)<br>+setCircunferencia(r: float): void<br>+getCircunferencia(): float<br>+area(): float<br>+perimetro(): float<br>+mostra(): void |

# Exercícios

- 2 Crie a classe ao lado como subclasse de Triangulo:
- ✓ O método `area()` deve retornar valor da área da circunferência, sabendo que
$$\text{area} = \text{base} * \text{altura}$$
  - ✓ O método `perimetro()` deve retornar o valor do perímetro:
$$\text{perimetro} = (\text{base} * \text{altura}) * 2$$
  - ✓ O método `mostra` deve exibir os valores de todos os atributos da classe

| Retangulo                      |
|--------------------------------|
|                                |
| +Retangulo(b: float, a: float) |
| +area(): float                 |
| +perimetro(): float            |
| +mostra(): void                |

# Exercícios

---

- 2 Instancie dois objetos na classe java principal, um da classe Circunferencia e outro da classe Retangulo, com os valores dos atributos digitados pelo usuário e utilize o construtor com parâmetros. Mostre os dados de cada objeto através do método mostra().



*That's all Folks!*